

Dynamic Memory Allocation in C++ (new, delete, Heap Mechanics)

[DEF] Definition to Remember

Dynamic memory allocation in C++ allocates heap memory at runtime using `new` and `new[]` operators, and releases memory using `delete` and `delete[]` operators to prevent memory leaks.

1. Concept Overview & Fundamentals

Dynamic Memory Allocation in C++ (new, delete, Heap Mechanics) provides modern high-performance capabilities combining low-level hardware control with high-level abstractions, object-oriented paradigms, and Standard Template Library (STL) utilities.

In modern C++ (C++11 through C++20):

* Ze

2. Technical Architecture & Memory Mechanics

C++ Runtime Execution Architecture



Modern C++ Feature Comparison

Modern C++ Paradigm	Legacy Equivalent	Engineering Advantage	Performance Impact
<code>**std::cin` / `std::cout`**</code>	<code>`scanf` / `printf`</code>	Type-safe, extensible for custom class	Buffered I/O (optimizable via <code>`ios::sy</code>
<code>**References (`Type&`)**</code>	Pointers (<code>`Type**`</code>)	Non-nullable, clean syntax without der	Direct alias; zero runtime overhead
<code>**std::vector<T>`**</code>	Raw Dynamic Array (<code>`malloc`</code>)	Automatic resizing, bounds safety, mem	Amortized <code>**O(1)**</code> <code>push_back</code> , contiguo
<code>**`nullptr`**</code>	<code>`NULL` / `0`</code>	Strongly typed pointer literal prevent	Zero overhead

3. Core Syntax & Implementation Patterns

Modern C++ Template Structure

* Quantitative Finance & HFT: High-frequency trading platforms use modern C++ for microsecond-level execution.

* Ga

Problem 1: calloc sentence

Problem Description & Objective:

C++ Code Implementation:

Implement

```
/*
 * Que
 * All
 */
/*
 * ==
 * COM
 * ==
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 * *
 *
 *
 *
 * ==
 */
#include
#include
```

```
// Use
using

//
int ma
in
//
ch

//
co
//
ci

//
co

//
de
dy

//
re
}
```

Problem 2: car malloc

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * All
 */

/*
 * ==
 * CON
 * ==
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 * *
 *
 *
 * ==
 */

#inclu
#inclu

// Use
using
```

```
struct
    st
    in
};

//
int ma
//
Ca

my
my

//
cc

//
de
my

//
re
}
```

Problem 3: float array malloc

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Dyn
 */

/*
 * ===
 * CON
 * ===
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 * *
 *
 *
 *
 * ===
 */

#include

// Use
using
```

```
//
int ma
in
//
cc
//
ci
//
if
//
fl
//
cc
fl
//
fo

}

//
cc
//
cc

//
de
an

//
re
}
```

Problem 4: point struct malloc

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Dyn
 */

/*
 * ==
 * COM
 * ==
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
```

```
 * *
 *
 *
 *
 * ===
 */

#include

// Use
using

struct
    in
    in
};

//
int ma
//
Po

p-
p-

//
co

//
de
p

//
re
}
```

Problem 5: realloc shrink

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Dem
 */

/*
 * ===
 * CON
 * ===
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 * *
```

```
*
*
*
* ===
* /

#include
#include

// Use
using

//
int ma
//
ve

//
co
//
fo

}

//
co

nu
nu

//
co
//
co
//
co
//
fo
//
co

//
re
}
```

6. Exam Scoring Points & Memory Revision

[MUST-WRITE] Must-Write Points

1. new operator allocates heap memory and calls the object constructor; delete calls destructor and frees heap.
2. Array allocation: `Type* arr = new Type[size];` must be paired with `delete[] arr`.
3. Always set pointers to `nullptr` after deletion to avoid dangling pointer references.
4. Memory leak prevention: Every new must have exactly one corresponding delete.
5. Modern C++ RAI: Smart pointers (`std::unique_ptr`, `std::shared_ptr`) automate dynamic memory lifecycle.

[RECALL] Quick Recall

new (Allocate + Construct) -> `nullptr` Check -> Use Object -> `delete[]` (Destruct + Free) -> `nullptr`